HybridKernel: A Pre-Registered Profiler Gate for Hybrid Attention/SSM Boundaries

Anonymous authors
Paper under double-blind review

Abstract

Hybrid attention/state-space language models invite a systems question: do attention-to-SSM layer boundaries create avoidable conversion, materialization, launch, or locality overhead that a fused boundary operator could remove? We report a Mac-feasible pre-GPU gate for this question, not a speed result. Architecture maps show large boundary-crossing activation streams, but a runtime audit weakens broad novelty because current vLLM hybrid SSM support already handles important state-layout and transfer paths. A threshold model says Granite 4.0 H needs roughly 25% genuinely avoidable boundary traffic at 60% recovery to clear a 3% proxy gain; Qwen3-Next needs 10.4% but is less directly matched to the Granite Mamba2 hypothesis. The artifact is a source-line audit, control feasibility matrix, fixed-request vLLM driver, native packet skeleton generator, pre-registered profiler-analysis parser, artifact-completeness verifier, and Mac-local Triton interpreter correctness for a toy boundary primitive. No throughput, latency, HBM, or GPU speedup claim is made.

1 Hypothesis

HybridKernel tests whether adjacent attention and SSM layers expose boundary-local overhead not already removed by serving systems. Candidate costs include layout conversion, residual-stream materialization, host launch gaps, cache-locality loss, or redundant memory traffic. The hypothesis is deliberately narrow: a useful kernel must compose with existing vLLM hybrid mechanisms rather than replace them.

2 Current Evidence

Artifact	Result	Decision
Architecture map	Granite has 8 boundaries and 20.0% boundary stream fraction; Qwen3-Next has 23 inferred boundaries and 47.9%.	Inspectable, not speed evidence.
Runtime audit	vLLM already handles hybrid state layout and transfer paths.	Broad novelty weakened.
Threshold model	Granite needs about 25.0% avoidable boundary traffic at 60% recovery for a 3% proxy gain.	Mac kernel work not justified.
Profiler driver	Fixed-request OpenAI-compatible driver dry-runs locally; its optional profile bracket calls vLLM `/start_profile` and `/stop_profile` around replay for dynamic Nsight capture. The runbook profiles the vLLM server, not just the HTTP client.	Native gate is less likely to be mis-run.
Analysis parser	JSON profiler summaries are reduced into avoidable-boundary share, recoverable-gain upper bound, median/IQR, and bootstrap confidence intervals. Rows must explicitly record matched control time, recoverable fraction, dtype, CUDA-graph state, batch/request shape, and control segment.	Decision rule is pre-registered and incomplete rows cannot inflate gains.
Artifact verifier	Native run directories are checked for metadata, server-side Nsight Systems and Compute scope, Nsight artifacts, separate Nsight server profiler logs, non-dry-run client replay JSON, readout rows, distinct repeated metric rows for the same model/config, and matching profiler-analysis outputs.	Client-only, warmup-only, dry-run, failed-request, duplicated-trace, stale-analysis, mixed-config repeat, and incomplete-log evidence is rejected.
Packet generator	A local script creates the required native run-packet skeleton with sentinel markers.	GPU operators get a consistent handoff; skeletons are rejected as evidence until filled.
Synthetic packet fixture	A Mac-local fixture documents the packet shape and exercises the checker with placeholder Nsight files.	Fixture is not profiler evidence.
Packet checklist	The native handoff now has a single required packet layout and stop-local-work decision.	Local readiness is saturated until GPU data exists.
Source/control audit	Source-line audit table and control feasibility matrix record what public surfaces were checked and which native controls are still placeholders.	Review hardening only; no proof of absence.
Triton correctness	The toy boundary primitive passes Phase 4 interpreter tests under the repo-local `triton-cpu` source install across block tails, fp16, non-contiguous inputs, and shape errors. Non-interpreter CPU-backend linking is environment-fragile on this Mac.	Kernel logic only; not CUDA or speed evidence.

Table 1: HybridKernel evidence before native GPU profiling.

3 Profiler Gate

The next experiment is a native NVIDIA/vLLM run with Nsight Systems and Nsight Compute. It must show a distinct boundary-local overhead attributable to attention/SSM transitions, not warmup, graph capture, batching, graph-state changes, dtype changes, or unrelated kernels. The runbook now specifies exactly how to reduce Nsight traces into parser fields: total step time, boundary-local time, matched non-boundary control time, recoverable fraction, dtype, CUDA-graph state, batch/request shape, and control segment. Promotion requires at least three repeated runs for the same model/config whose recoverable-gain upper bound clears 3%. The native controls are Granite 4.0 H as the closest target, a same-request Transformer-heavy control, a mostly-SSM/Mamba control if available, and warmup/graph-capture rejection. The analysis parser computes

$$\text{gain}_{ub} = \frac{\max(\text{boundary ms} - \text{matched control ms}, 0)}{\text{total step ms}} \times \text{recoverable fraction}.$$

If repeated native summaries show less than 1% recoverable gain, the branch should be killed or shelved. Before any run is cited, the artifact verifier must also pass on the exact

run directory, confirming environment metadata, server-side Nsight Systems and Nsight Compute scope, profiler artifacts, separate Nsight server profiler logs, non-dry-run client replay JSON with all request statuses marked `ok`, the pre-registered readout questions, at least three distinct repeated metric rows for the same model/config, and profiler-analysis outputs that match the same metric file. A packet generator creates this directory shape for GPU operators, but the checker rejects skeleton sentinels until real metadata, metrics, and readout entries replace them. This is an admissibility check only; it does not promote the method.

4 Claim Boundary

Allowed now: HybridKernel is a weakly alive profiler-driven systems branch with a runnable native measurement plan, source/control audit artifacts, and Mac-local interpreter correctness for one toy boundary primitive. Not allowed: throughput, latency, HBM, energy, occupancy, or vLLM speedup claims. The toy primitive is intentionally insufficient as the proposed systems kernel; it only checks indexing and Triton plumbing. If native profiling does not reveal separable boundary overhead, the branch should be killed.

5 Artifacts

- `experimental/hybridkernel/phase2/nvidia_vllm_profiler_runbook.md`
- `experimental/hybridkernel/phase2/profiler_driver.py`
- `experimental/hybridkernel/phase2/create_native_run_packet.py`
- `experimental/hybridkernel/phase2/analyze_profiler_metrics.py`
- `experimental/hybridkernel/phase2/check_profiler_run_artifacts.py`
- `experimental/hybridkernel/phase2/native_run_packet_checklist.md`
- `experimental/hybridkernel/phase1/source_line_audit_table.md`
- `experimental/hybridkernel/phase2/control_feasibility_matrix.md`
- `experimental/hybridkernel/phase2/mac_reproducibility_command.md`
- `experimental/hybridkernel/phase2/profiler_analysis_gate.md`
- `experimental/hybridkernel/phase2/pre_gpu_threshold_model.md`
- `experimental/triton_cpu_source_install_20260506.md`
- `experimental/hybridkernel/paper/colm_workshop_scaffold.md`
- `experimental/hybridkernel/paper/reviewer_pack.md`